PROGRAMMING & CS FUNDAMENTALS

Compilers vs Interpreters Visual Guide

From source text to running program — lexing, parsing, and the JIT middle ground.

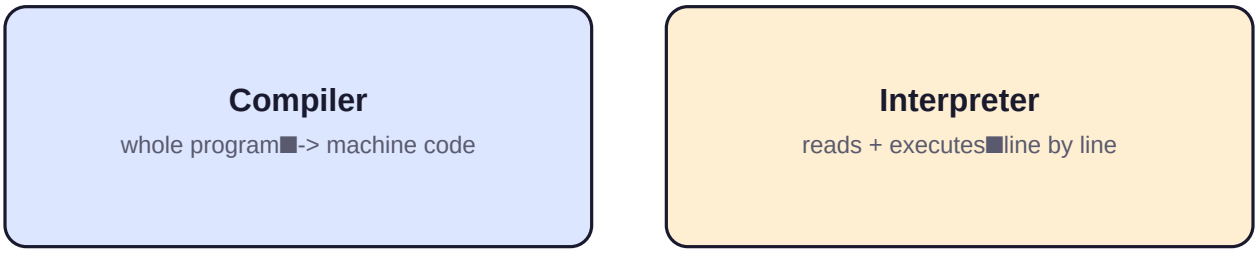
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Compiler vs Interpreter

Translate-then-run versus run-as-you-read.



Compiler: slow build, FAST run (C, Rust, Go). Errors surface before shipping.
Interpreter: instant start, slower run (Python, Ruby). Errors surface at runtime.

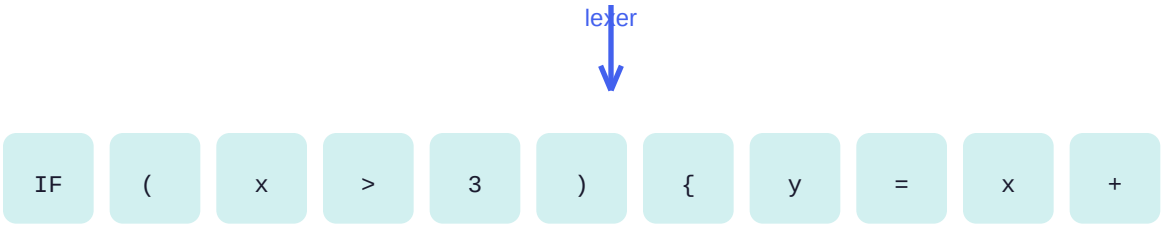
- It's a spectrum: bytecode VMs (JVM, CPython) compile to IR, then interpret THAT.
- Ahead-of-time (AOT): compile once, ship binary. JIT: compile hot code at runtime.
- Transpilers (TypeScript -> JS) are compilers whose target is another language.

STEP 2/6

Lexing: Text to Tokens

The scanner chunks characters into meaningful words.

```
if (x > 3) { y = x + 1; }
```



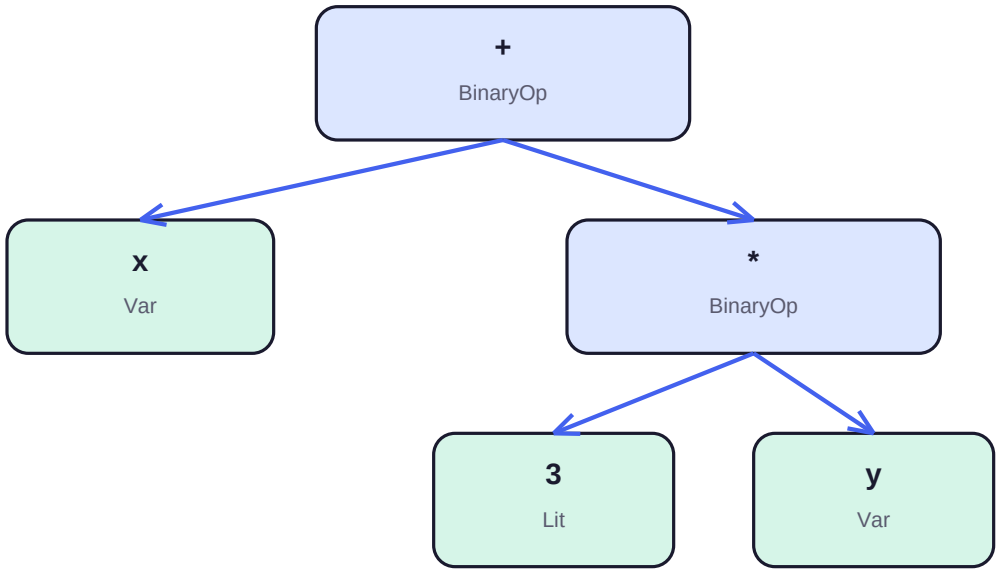
- Tokens: keywords, identifiers, literals, operators, punctuation — whitespace dies here.
- Regex-powered: each token type is a regular expression matched longest-first.
- Lex errors are the friendliest: 'unexpected character @' points at the exact spot.

STEP 3/6

Parsing: Tokens to AST

Grammar rules assemble tokens into a tree of meaning.

```
tokens:  x  +  3  *  y
```



Precedence encoded in tree SHAPE: * binds tighter, so it sits deeper.

- AST drops punctuation: no parens or semicolons — pure structure.
- Recursive descent: one function per grammar rule — hand-written, readable.
- Parse errors: 'expected ; at line 12' — the parser knows the grammar you violated.

STEP 4/6

IR & Optimization Passes

Lower to a simple intermediate form, then rewrite it smarter.

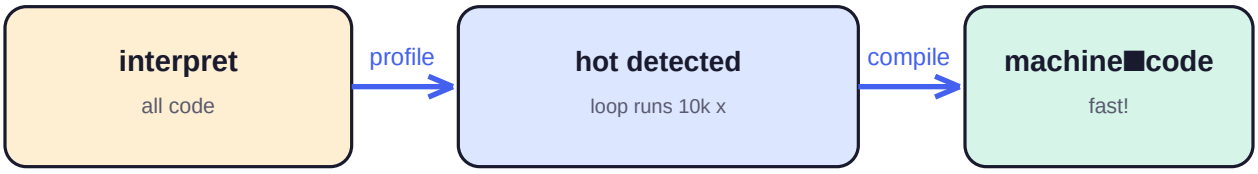
```
# source:   result = (a + b) * 2 + (a + b)
# IR:       t1 = a + b
#           t2 = t1 * 2
#           result = t2 + t1
# passes: constant folding, CSE, dead-code elim, inlining, loop unroll
```

- IR (LLVM, JVM bytecode): one backend per CPU, one frontend per language — N*M becomes N+M.
- Common subexpression elimination: compute (a+b) once, reuse t1.
- Optimizers preserve SEMANTICS: -O3 can reorder, but never change what the program means.
- SSA form: each variable assigned once — makes dataflow analysis tractable.

STEP 5/6

JIT: Compile the Hot Path

Interpret first, profile, then compile what actually runs hot.




```
# tiered JIT (JVM, V8):
# tier 0: interpret  ->  tier 1: quick baseline JIT
# -> tier 2: optimizing JIT with speculative inlining
# deopt: assumption broke? fall back to interpreter, re-profile
```

- Speculation: 'this is always an int' — compile for ints, guard, deopt on surprise.
- Warmup cost: JITs are slow for short scripts — AOT wins CLIs, JIT wins servers.
- PyPy, V8, HotSpot: same playbook, different decades.

STEP 6/6

Compiler Wisdom

What the pipeline teaches you as a programmer.

- Read error messages bottom-up: lex errors (characters), parse errors (grammar), type errors (meaning).
- Write JIT-friendly code: stable types, small functions, predictable loops.
- Undefined behavior (C/C++): the optimizer ASSUMES it never happens — and deletes your 'safety' checks.
- Interpreted doesn't mean slow: PyPy/JIT and native extensions close the gap for hot code.
- Learn one IR (LLVM): it demystifies every 'magic' optimization you've ever benchmarked.
- Syntax is the easy 10% — name resolution, types, and codegen are the real compiler.

STEP ✓I✓

Key Takeaways

Compilers vs Interpreters — Remember

- Pipeline: lex (tokens) -> parse (AST) -> IR -> optimize -> codegen; each stage catches its own errors.
- Compilers: slow build, fast run; interpreters: instant start, slower run; bytecode VMs sit between.
- IR decouples frontends from backends; SSA + optimization passes rewrite without changing meaning.
- JIT profiles real execution, compiles hot paths with speculation, deopts on surprise.
- Write JIT-friendly code: stable types, small functions — and respect undefined behavior.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev